My main research goal is to build large neural models that are applicable to real-world scenarios, by addressing the inherent limitations of current neural models. Specifically, I am interested in (1) allowing neural models to be lifelong learners [5, 3, 1] (2) providing privacy and security guarantees for neural models [6], and (3) enabling neural models to follow human instructions [4, 8, 7, 2].

**Developing Large Neural Models to be Lifelong Learners** During my internship, I intend to develop truly *lifelong* learners that can continue to accumulate world knowledge without forgetting just as humans do. Large neural models obtain a vast amount of world knowledge during initial pretraining. However, fully parametric approaches suffer from having very little control over **updating** the knowledge stored in the models, which may be critical for models deployed in the real world (e.g. QA about real-world events). I plan to develop methods that allow updating parametric knowledge in neural models in a compute-efficient manner.

Updating the knowledge stored in the implicit parameters is non-trivial considering how they are acquired in the first place. The knowledge is not explicitly accumulated or specified such as in non-parametric knowledge bases (KBs), but implicitly gained from the pretraining objective which enables latently encoding the knowledge in the parameters when used to pretrain on large amounts of unlabeled data. As initial steps for developing truly lifelong learners, I have explored continuing the pretraining objective that allowed it to gain the knowledge in the first place with continual learning methods and explore which factors help models gain and update new knowledge without forgetting the past knowledge [5]; I found that freezing the original parameters and updating new parameters via adapters results in a good trade-off between gaining new knowledge and forgetting. I have also explored an efficient method of updating Pretrained Language Models (PLMs) by utilizing recent dumps of Wikipedia and Wikidata for training and evaluation, respectively [3]. I found that utilizing the *diff* of Wikipedia dumps (only data that are updated or added) is much more efficient than updating with the entire snapshots, especially when updating with continual learning methods.

How can we supply external knowledge without concatenating it in-context all the time? For example, there may be informative prompts that give some external knowledge to models; it may be much more efficient if we *inject* the prompt instead of explicitly attending to it every single time. One example of this case may be a persona of a chatbot agent which is always fixed and concatenated to the history of utterances in order to condition the agent to possess the given persona. In these cases, we explored injecting the prompt [1] which parameterizes the external knowledge (persona) using a teacher model, allowing much more efficient inference during deployment. We show that this approach is highly beneficial in scenarios where the prompt is extremely long and is constantly appended.

I believe there exists an efficient method of updating large neural models, just as humans learn to update and add to our accumulated knowledge. More specifically, I believe there exist methods that can gain new knowledge while minimizing the forgetting of already-obtained knowledge. One interesting approach that I want to explore is utilizing reinforcement learning (RL) to allow neural models to *select* the knowledge they need to update as the world changes. For example, developing RL agents that can browse through the internet and determine whether to concatenate the retrieved knowledge in-context (temporary knowledge) or inject the knowledge (long-term knowledge) is an interesting future work. This, however, requires RL algorithms that can generalize to new environments (as the world changes), which I believe remains an open-ended problem in RL as well, which makes it an interesting problem to delve into for a thesis.

**Memorization and Privacy in Large Neural Models** If we view large neural networks as KBs, one interesting question is how we can perform operations as we would perform in traditional KBs. For example, large neural models may memorize unwanted information during initial pretraining. In this scenario, we may want to *delete* some specific knowledge stored in models such as sensitive information of individuals. In our recent work [6], we show that simply performing gradient *ascent* is effective at forgetting target information without hindering the general Language Model (LM) capabilities. Surprisingly, it sometimes results in a boost of LM capabilities, which we are further exploring in an ongoing work [9].

While we made some initial steps [1, 6] in performing fine-grained knowledge operations with large neural models, there is more to be done in interacting with LMs the way we do with traditional KBs.

One thing that I want to explore regarding this research direction is figuring out which aspect of neural models contributes to memorization capabilities. For example, I am very curious about what the role of the training objective (e.g. gradient descent/ascent, causal/masked language modeling) is in determining the granularity of the knowledge being stored in the parameters. Next, it would be interesting to utilize the findings in developing LMs that are capable of controlling its fine-grained knowledge, while still being fully parametric.

**Language Models that Perform New Tasks at Test Time via Instructions** Another research direction I am interested in is developing LMs that can perform new tasks at test time. Previous works have explored continually adapting Pretrained Language Models (PLMs) through multitask training on multiple downstream tasks together with a text description (instruction) of the task, and show that it helps perform tasks at test time that were not seen during the training phase. I would like to develop LMs that can follow the given text instructions by approaching the problem with a totally different perspective.

One interesting approach we explored is *flipping* the input and output during multitask training; we showed it to be effective at generalizing to novel output options (labels), resulting in a significant performance improvement on unseen tasks [7]. We also explored replacing the multitask training phase with a modular approach consisting of two phases [8, 2]. Phase 1 involves training *experts* for each specific task during training while Phase 2 involves searching for the relevant expert to perform the given task. This modular approach of training and retrieving multiple experts instead of a single expert has many advantages such as learning new tasks with minimal additional of costs. An interesting extension of this work is *merging* the experts to develop efficient meta-experts, capable of performing different granularity of the given tasks.

Further extending this capability, developing LMs that can truly follow instructions and act in the external world seems like an interesting application of LMs. As large LMs become more capable, they are further utilized for actual decision-making in the real world such as planning and reasoning the actions of robots or external agents. In order to further endow more responsibilities, we have to make sure that they truly follow and understand the given instructions. Current large LMs are far from truly understanding the prompts as shown in their incapability of understanding the concept of negation [4]. Driving the community to carefully consider the major shortcomings of large LMs before distilling major decision-making responsibilities on them and developing methods to allow them to truly reason and plan when given human instructions is another interesting research direction that I would like to explore further.

**Future Plans at University of Washington** I believe the UW CSE program is a great fit for me. I look forward to working with Luke Zettlemoyer and Noah A. Smith who both share the research interest in developing novel language models that are more efficient. I also look forward to working with Yejin Choi and Hannaneh Hajishirzi on developing general-purpose neural models and neural models that can plan and reason. Lastly, I highly value the collaborative and inclusive environment of the UW CSE program which I believe will help me expand my research perspective and conduct highly impactful research.

# References

[1] Eunbi Choi, Yongrae Jo, **Joel Jang**, and Minjoon Seo. Prompt injection: Parameterization of fixed inputs. ***Under Review*** *at ICLR 2023*. Openreview link.

[2] **Joel Jang**, Seungone Kim, Seonghyeon Ye, Kyungjae Lee, Moontae Lee, and Minjoon Seo. Exploring the benefits of training expert language models over instruction tuning. ***To Be Submitted*** *to ICML 2023*.

[3] **Joel Jang**, Seonghyeon Ye, Changho Lee, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun Kim, and Minjoon Seo. Temporalwiki: A lifelong benchmark for training and evaluating ever-evolving language models. *EMNLP 2022*.

[4] **Joel Jang**, Seonghyeon Ye, and Minjoon Seo. Can large language models truly understand prompts? a case study with negated prompts. *NeurIPS 2022 Workshop on Transfer Learning for NLP*.

[5] **Joel Jang**, Seonghyeon Ye, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun Kim, Stanley Jungkyu Choi, and Minjoon Seo. Towards continual knowledge learning of language models. *ICLR 2022*.

[6] **Joel Jang**, Dongkeun Yoon, Sohee Yang, Sungmin Cha, Moontae Lee, Lajanugen Logeswaran, and Minjoon Seo. Knowledge unlearning for mitigating privacy risks in language models. ***Under Review*** *at ICLR 2023*. Openreview link.

[7] Seonghyeon Ye, Doyoung Kim, **Joel Jang**, Joongbo Shin, and Minjoon Seo. Guess the instruction! making language models stronger zero-shot learners. ***Under Review*** *at ICLR 2023*. Openreview link.

[8] Seonghyeon Ye, **Joel Jang**, Doyoung Kim, Yongrae Jo, and Minjoon Seo. Retrieval of soft prompt enhances zero-shot task generalization. ***To Be Submitted*** *to ACL 2023*.

[9] Dongkeun Yoon, **Joel Jang**, Sungdong Kim, and Minjoon Seo. Gradient ascent makes better language models. ***To Be Submitted*** *to ACL 2023*.